

ESP32 Smart Voltage Protector
FSMB Recruitment 2026, Phase 1 - Embedded Systems Engineer
Md Mahmud Akon - 10 October 2026

Design approach

For the required Project assessment, I built an ESP32-based device that disconnects an appliance during unsafe voltage or current conditions. I developed the protection and relay control first, added the local settings interface, and then added MQTT and the web dashboard. The complete project was tested in Wokwi.

Wokwi does not provide a real AC grid or the exact voltage or current sensors needed in this project, so I used two potentiometers as sensor emulators. They represent conditioned 0-3.3 V outputs from isolated voltage and current sensors. The firmware converts them to 150-300 V RMS and 0-10 A RMS. A relay module represents the contactor, and a green LED represents the protected appliance.

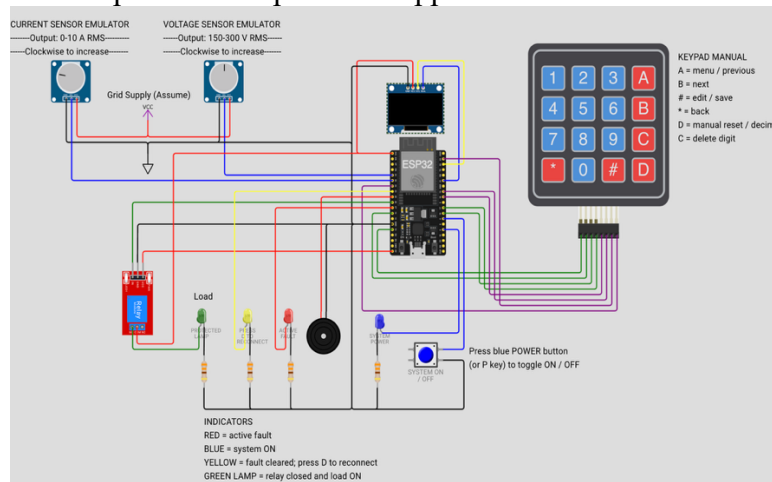


Figure: Wokwi wiring diagram: ESP32, sensor emulators, keypad, OLED, relay, and indicators.

Operation and control

The ESP32 continuously check V_{min} , V_{max} , I_{max} , and sudden voltage rise. If any of the parameters cross the limit the relay opens, the **green load LED** turns off, and the **red fault LED** and buzzer turn on.

When the readings return to normal value, the relay remains open and the **yellow LED** asks for a manual reset. A reset is rejected while a fault is active. After a valid reset, the readings must stay healthy for three seconds (by default) before the relay closes. I chose this method because reconnecting after only one normal reading could be unsafe for the Load.

A keypad and OLED is used for local settings and showing the status. Any invalid settings are rejected and only accepted settings are stored in ESP32 flash memory. The MQTT dashboard shows live values, relay state, graphs, and recent activity logs. Using the dashboard settings can be updated and request a safe reset. Protection system runs locally, so an MQTT failure can not stop the ESP32 from opening the relay.

Results and limitations

I manually tested normal operation, over-voltage, under-voltage, over-current, sudden rise, fault recovery, unsafe reset, local and remote/computer settings, system power, and MQTT failure. The relay, indicators, OLED, Serial Monitor, and dashboard showed the expected states. The firmware also compiled successfully for the ESP32.

This is a simulation, not a mains-ready product. The potentiometers do not model real sensor noise or isolation, and the public MQTT broker is only for a demonstration. Real hardware would need isolated sensing, fusing, ADC protection, safe clearances, and a correctly rated relay or contactor.